CHAPTER 7

MAKING CONTROL FLOW EASY TO READ



Group 09 Members:

- Heng Menghong
- Chea Sophaktra
- Chheang Udomveasna
- Vandeth Sothy

1. The Main Idea

Readable code helps people understand what happens without extra mental effort.

Make the reader’s path through the code feel natural.

CLEAR

Use familiar, direct expressions.

FLAT

Avoid unnecessary nesting and jumps.

PREDICTABLE

Keep execution flow easy to follow.

Goal: reduce the time needed to understand the code, not simply the number of lines.

2. Write Conditions Naturally

Put the changing value first and the stable comparison value second.

LESS NATURAL

```
if (10 <= length) {
    ...
}
```

MORE NATURAL

```
if (length >= 10) {
    ...
}
```

Why it helps

The reader sees the variable being checked first.

The condition reads like a sentence.

3. Choose a Clear if/else Order

There is no single rule for every case; choose the order that makes the logic easiest to grasp.

EXAMPLE: NORMAL CASE FIRST

```
if (expand_all) {
    expand_items();
} else {
    render_items();
}
```

Consider showing first:

The positive or expected case

The simpler branch

The more important case

An error or exceptional case, when it deserves attention

Pick the order that best matches the reader's expectations.

4. Use Ternary Expressions Carefully

The ?: operator is useful for short, simple choices, but can obscure complicated logic.

SIMPLE CHOICE: TERNARY WORKS

```
period = "pm" if hour >= 12 else "am"
```

COMPLEX CHOICE: PREFER IF/ELSE

```
if (exponent >= 0) {
    result = base * power(exponent);
} else {
    result = 1 / power(base, -exponent);
}
```

Clarity matters more than saving a line.

5. Keep Loop Behavior Obvious

Prefer a straightforward while loop when it expresses the intended behavior clearly.

WHILE LOOP

```
while (condition) {
    do_work();
}
```

DO/WHILE LOOP

```
do {
    do_work();
} while (condition);
```

Important difference

While Loop
Checks the condition before execution.
Runs only if the condition is true.
May execute zero times.

Do/While Loop
Executes the body before checking the condition.
Repeats if the condition is true.
Always executes at least once.

6. Use Early Returns for Special Cases

Guard clauses can keep the main logic from being buried inside conditions.

GUARD CLAUSES

```
if (!user) return;
if (!user.has_permission) return;

show_dashboard();
```

Benefits

Special cases are handled early.

The normal path stays visible.

Fewer indentation levels are needed.

Caution: in languages such as C, make sure resources are cleaned up correctly.

7. Avoid Unnecessary Jumps

goto can make execution flow difficult to trace, though limited cleanup uses may be practical in C.

PREFER STRUCTURED FLOW

```
check_input();
process_data();
return;
```

BE CAREFUL WITH JUMPS

```
if (error) goto cleanup;
...
cleanup:
release_resources();
```

Remember

Many jumps can create “spaghetti code.”

Use structured alternatives whenever they remain clear.

A single cleanup label can be reasonable in some C programs.

8. Minimize Nesting and Follow the Flow

Deep nesting makes readers keep track of too many active conditions.

NESTED LOGIC

```
if (user) {
    if (user.has_permission) {
        show_dashboard();
    }
}
```

FLATTER LOGIC

```
if (!user) return;
if (!user.has_permission) return;
show_dashboard();
```

For loops, continue can skip irrelevant items early:

EXAMPLE

```
for student in students:
    if not student.is_eligible:
        continue
    process(student)
```

FINAL CHECKLIST

Can someone follow the code easily?

- Is the condition written in a natural order?
- Is the if/else order easy to predict?
- Is a ternary expression genuinely simple?
- Does the loop’s execution count make sense?
- Can guard clauses reduce nesting?
- Are jumps and branches easy to trace?

Write code for the next person who has to read it.